

Military Asset Management System

1. Project Overview

The Military Asset Management System is a secure, role-based operational platform for tracking military equipment, purchases, transfers, assignments, expenditures, base readiness, and audit activity. It is designed for command and logistics operations where visibility, accountability, and access control are critical.

This project combines a Java Spring Boot backend, a React + Vite frontend, and a MySQL database. It supports multi-user operations with different permission levels and provides a live dashboard for operational oversight.

Assumptions

- The system is intended for internal military command operations.
- Assets are tracked per equipment type and per base.
- Inventory can move across bases through transfers.
- Audit events must be recorded for operational accountability.
- Access must be restricted based on role and base assignment.

Limitations

- Historical opening balance is approximate by design and should not be treated as exact historical ledger data. This is a documented limitation of the current implementation.
 - The project is a demo-style operational management solution and is not a full enterprise financial ledger system.
-

2. Tech Stack & Architecture

Frontend

- React.js
- Vite
- JavaScript
- Axios for API communication
- CSS-based custom UI styling
- React Router for navigation

Backend

- Java 17
- Spring Boot 3.3.4
- Spring Web
- Spring Data JPA
- Spring Security
- JWT authentication
- Bean Validation
- MySQL driver

Database

- MySQL
- JPA-managed entity persistence with Hibernate

Architecture Summary

- Frontend handles user interaction, navigation, and operational dashboards.
 - Backend exposes REST APIs and enforces RBAC.
 - MySQL stores all operational records and relationships.
 - JWT ensures stateless authenticated sessions.
 - Audit logging records actions such as login, asset creation, transfer cancellation, and user updates.
-

3. Data Models / Schema

The following core entities represent the system data model.

3.1 User

Represents authorized system users.

Fields:

- id
- name
- email
- password (stored hashed)
- role
- base
- active
- createdAt
- updatedAt

Relationships:

- Many users can belong to one base.
- Each user has one role.

3.2 Base

Represents a military base or operational site.

Fields:

- id
- name
- code
- location
- active
- createdAt

- updatedAt

Relationships:

- One base has many assets.
- One base has many users.

3.3 EquipmentType

Defines the category of equipment tracked.

Fields:

- id
- name
- category
- description

Relationships:

- Many assets belong to one equipment type.
- Many purchases, transfers, and expenditures reference one equipment type.

3.4 Asset

Represents the current inventory record for a specific equipment type at a base.

Fields:

- id
- assetCode
- equipmentType
- base
- quantity
- status
- createdAt
- updatedAt

Status values include:

- ACTIVE
- DAMAGED
- MISSING

3.5 Purchase

Represents procurement or acquisition activity.

Fields:

- id
- base

- equipmentType
- quantity
- unitPrice
- purchaseDate
- referenceNumber
- notes
- createdBy

3.6 Transfer

Represents movement of stock from one base to another.

Fields:

- id
- fromBase
- toBase
- equipmentType
- quantity
- transferDate
- referenceNumber
- status
- createdBy

Status values include:

- COMPLETED
- CANCELLED

3.7 Assignment

Represents personnel assignment of asset stock to an individual or operational role.

Fields:

- id
- asset
- base
- personnelName
- quantity
- assignmentDate
- status
- createdBy

3.8 Expenditure

Represents consumption or usage of resources.

Fields:

- id

- base
- equipmentType
- quantity
- expenditureDate
- reason
- createdBy

3.9 AuditLog

Tracks system actions and security-sensitive events.

Fields:

- id
- user
- action
- entityType
- entityId
- metadata
- timestamp

4. RBAC Explanation

Role-based access control is enforced through Spring Security with `@PreAuthorize` checks and base-scoping logic.

Roles

ADMIN

- Full system access.
- Can view all bases, assets, purchases, transfers, users, equipment types, and audit logs.
- Can create, update, and manage records across the platform.

BASE_COMMANDER

- Can access records for their assigned base only.
- Can manage assets and operational records connected to that base.
- Cannot access other bases or full admin-only data.

LOGISTICS_OFFICER

- Can access operations relevant to logistics tasks.
- Can create and manage purchase and transfer records within allowed scope.
- Restricted from admin-level controls and access to all system data.

Enforcement Method

The app uses:

- `@PreAuthorize` on controller methods
- `BaseScopeService` to determine effective base access
- user-to-base mapping to restrict unauthorized access
- validation before creating or updating records linked to a base

Example logic:

- If a base commander requests access to another base, the request is denied.
 - If a transfer or asset operation does not belong to the user's base scope, the API rejects the request.
-

5. API Logging

The application records operational actions in the audit log using the `AuditService` layer.

Logged actions include

- LOGIN
- CREATE_ASSET
- UPDATE_ASSET
- DEACTIVATE_ASSET
- CREATE_PURCHASE
- UPDATE_PURCHASE
- CREATE_TRANSFER
- CANCEL_TRANSFER
- CREATE_ASSIGNMENT
- CREATE_BASE
- UPDATE_BASE
- CREATE_USER
- UPDATE_USER
- CREATE_EQUIPMENT_TYPE
- UPDATE_EQUIPMENT_TYPE

Logging behavior

Each audit log stores:

- actor user
- action name
- entity type
- entity id
- metadata (such as code, reference number, or message)
- timestamp

This gives system administrators a chronological record of operational and security-sensitive activity.

6. Setup Instructions

6.1 Prerequisites

- Java 17+
- Maven
- MySQL server
- Node.js and npm

6.2 Database Setup

Create a MySQL database named:

`military_asset_management`

Set environment variables before starting the backend:

```
export DB_USERNAME=root
export DB_PASSWORD=your_mysql_password
export JWT_SECRET=your_secure_secret_key
```

6.3 Backend Setup

```
cd backend
mvn spring-boot:run
```

The backend runs on:

`http://localhost:8080`

6.4 Frontend Setup

```
cd frontend
npm install
npm run dev
```

The frontend runs on:

`http://localhost:5173`

6.5 API Base URL

```
http://localhost:8080/api
```

7. Working Login Credentials

The application seeds demo users on first startup. The valid login accounts are:

- Admin: `admin@military.local` / `admin123`
- Base Commander: `commander@military.local` / `commander123`
- Logistics Officer: `logistics@military.local` / `logistics123`

These credentials are used for demo and operational testing in the local environment.

8. Key API Endpoints

Authentication

- `POST /api/auth/login`
- `POST /api/auth/register`
- `GET /api/auth/health`

Dashboard

- `GET /api/dashboard`
- `GET /api/dashboard/trends`
- `GET /api/dashboard/movement`

Assets

- `GET /api/assets`
- `POST /api/assets`
- `PUT /api/assets/{id}`
- `DELETE /api/assets/{id}`

Purchases

- `GET /api/purchases`
- `POST /api/purchases`
- `PUT /api/purchases/{id}`

Transfers

- `GET /api/transfers`
- `POST /api/transfers`
- `POST /api/transfers/{id}/cancel`

Assignments

- `GET /api/assignments`
- `POST /api/assignments`

Bases

- `GET /api/bases`
- `POST /api/bases`
- `PUT /api/bases/{id}`
- `GET /api/bases/{id}/summary`

Personnel

- `GET /api/users`
- `POST /api/users`
- `PUT /api/users/{id}`

Equipment Types

- `GET /api/equipment-types`
- `POST /api/equipment-types`
- `PUT /api/equipment-types/{id}`

Audit

- `GET /api/audit-logs`
-

9. Main Features

- Secure JWT login
 - Role-based access control
 - Base-scoped visibility
 - Asset inventory tracking
 - Purchase tracking
 - Transfer management
 - Assignment tracking
 - Expenditure tracking
 - Dashboard summary and trend analytics
 - Audit trail visibility for admins
-

10. Operational Notes

- The frontend and backend are designed to run together for a complete operational demo.
 - The backend seeds sample records on the first application startup.
 - MySQL is the default persistence layer.
 - The system uses a single-page frontend with route-based sections for dashboard and resource tables.
-

11. Final Known Limitation

The current historical opening-balance implementation is intentionally limited and should be treated as an approximate operational indicator rather than a precise historical ledger value.

This is an accepted limitation of the project design and remains documented here.